4.1 Discrete Prompt

Recent works [11, 110] allow LLMs to quickly adapt to unseen domains by discrete prompting, and GPT-3 [11] is the first work that introduces how to perform an unseen task using an LLM with zero-shot/few-shot discrete prompts without updating the LLM's inner parameter. We give a formal definition of the discrete prompt framework below.

Problem Setup. Given an LLM $f_\Theta(\cdot)$ where Θ denotes pre-trained model parameters, the task is to elicit desired output $\boldsymbol{y}$ from LLM with a discrete prompt p and a test query, denoted as $\boldsymbol{y} = f_\Theta([p; \boldsymbol{c}])$, when freezing Θ. It is worth noting that both $\boldsymbol{y}$, p and $\boldsymbol{c}$ are sequences of tokens (i.e., natural language sentences). The rationale behind using discrete prompts is that they can serve as instructions to elicit the generalized reasoning abilities of LLMs. By following such instructions, LLMs can perform domain-specific tasks that they have not been specifically trained for. This approach allows LLMs to demonstrate their ability to apply previously learned knowledge to new and diverse situations, thus enhancing their overall effectiveness and utility.

Depending on the prompting types, discrete prompts can be divided into two categories: (1) zero-shot [7, 68, 134], where the prompt p consist of only the task description; and (2) few-shot [90, 102, 167], where the prompt p consists of the task description and few illustrative examples. The key difference between zero-shot and few-shot prompts is whether or not illustrative examples are provided.

4.1.1 Zero-shot Discrete Prompts. The zero-shot setting represents the cold-start scenario, where not a single supportive labeled example is available.

Figure 4 presents a toy example of how zero-shot discrete prompts work. The task description that compromises the prompt p can be curated by human users or automatically generated by templates, where the intent of the task and the expected outcomes are described in natural language. However, as stated in [136], post-process is sometimes required to extract the rigorous prediction results from the unbounded raw outputs. Researchers demonstrate that instruction alignment pre-training enables decent zero-shot performance on various unseen tasks [11, 134, 165], where different tasks can be represented in a unified sequence generation format. PADA [7] is one of the pioneering works that explore how to elicit the domain adaptation ability of LLMs for domains unseen during the training phase. PADA first generates the target domain name followed by a set of domain-related features related to the test query, and then uses them together as the prompt to predict task labels. Follow-up works explore how to utilize zero-shot discrete prompt for domain adaptation in sentiment analysis [60], image classification [38], semantic segmentation [35], and rumor detection [82]. Later, Kojima et al. [68] extend the few-shot-Chain-of-Thoughts(Few-shot-CoT) [167] into zero-shot-CoT to elicit multi-step reasoning ability of LLMs. The core idea of Zero-shot-CoT is a two-stage prompting, where the 1st stage simply adds the same prompt "*Let's think step by step*" before each answer to derive the reasoning process sentences, and the 2nd stage takes the generated reasoning sentences to generate the final answer. Zero-shot-CoT has achieved

Task description

Please determine if the two sentences entail, contradict, or are neutral to each other.

Test query

Premise: She emerged vigorous with Apgar of 7 and 8.

Hypothesis: She had low APGAR scores.

Answer:

LLM response

LLM: Contradiction

Fig. 4. An example (adapted from [70]) of zero-shot discrete prompts, where task description, and/or a test query are provided to LLMs. No illustrative examples are provided in zero-shot prompts.

significantly stronger performance than the standard zero-shot prompting method on arithmetic, symbolic reasoning, and other logical reasoning tasks.

4.1.2 Few-shot Discrete Prompts. The few-shot setting reflects the characteristics of sparse training samples of many domain-specific applications (i.e., only a few annotated examples are available).

Figure 5 presents a toy example of how few-shot discrete prompts work. Different from zero-shot prompts, a few examples that further convey the task intention and provide illustrations of the desired output formats are included in the prompt p. Researchers have observed that few-shot prompts yield more stable output formats and more decent performance on downstream tasks [11, 110]. Chain-of-Thought (CoT) [167] improves the domain specialization ability of LLMs by introducing a series of intermediate reasoning steps for complex reasoning tasks, but it also brings extra cost in manually designing CoTs for each test example. As a follow-up, Auto-CoT [184] eliminates manual designs by appending the "*Let's think step by step*" prompt to the given task context and letting LLMs generate reasoning chains directly. Other than the natural language format instructions, CoCoGen [90] studies the programming language format instructions to tackle structured reasoning tasks.

More advanced techniques are then proposed to further improve discrete instruction of LLMs for domain specialization and customization. For instance, *ensemble-based instruction* [89, 159, 160] utilizes multiple different instructions to derive multiple model outputs and then aggregates these outputs to achieve better task performance. Another line of research proposes *recursive instruction* [2, 32, 33, 67, 172, 187] to breaks down a complex unseen task into a series of subtasks that are relatively easier to solve and then employs LLMs with specific instructions for each subtask.

Task description
Please determine if the two sentences entail, contradict, or are neutral to each other. Below are several examples.
Example 1
Premise: ALT, AST, and lactate were elevated as noted above.
Hypothesis: The patient has abnormal lfts.
Answer: Entailment
Example 2
Premise: Chest x-ray showed mild congestive heart failure.
Hypothesis: The patient complains of cough.
Answer: Neutral

Test query
Premise: She emerged vigorous with Apgar of 7 and 8.
Hypothesis: She had low APGAR scores.
Answer:
LLM response
LLM: Contradiction

Fig. 5. An example (adapted from [70]) of few-shot discrete prompts, where task description, illustrative examples, and/or a test query are provided to LLMs. Compared to zero-shot prompts, a few supportive labeled samples are provided in few-shot prompts.

4.1.3 Open Challenges. Utilizing discrete prompts helps LLMs leverage their inherent knowledge to adapt to new and diverse situations. This approach not only demonstrates the flexibility and adaptability of LLMs but also enhances their overall effectiveness and utility across a wide range of domains and tasks. However, crafting discrete prompts of LLMs for domain specialization poses several open challenges:

(1) *Effectiveness*: Often the discrete instructions are curated by domain experts or follow some types of templates. It is arguable whether the instructions used are the most effective ones. Therefore, there is a need for evaluation

of these instructions. This can be achieved through collaboration between domain experts and data scientists, who can analyze the performance of the LLMs and adjust the instructions accordingly. An automatic evaluation would be even better.

(2) *Scalability and adaptability*: Automated ways to generate and select/combine discrete instructions without excessive human intervention is another promising direction to improve discrete instructions of LLMs.

4.2 Continuous Prompt

Similar to discrete prompts, the continuous prompt is a sequence of tokens proposed to attach with the input sentence and guide LLMs with extra knowledge but can be learned from the downstream dataset by continuous *prompt tuning*. In this case, the continuous prompt serves as a *soft* parameterized prompt instead of the hard-coded instruction as discrete language phrases. Prompt tuning is to optimize the prompt that adapts an LLM to customized tasks or domains with the preservation of LLM's general language understanding ability. Here, one can solely update prompt-related parameters, whose quantity is around only 0.01% of the total number of the LLM's parameters, while freezing the LLM itself during the fine-tuning phase.

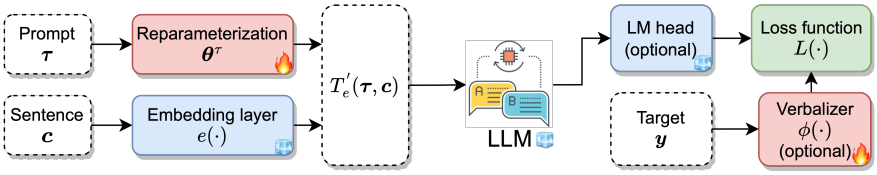

Fig. 6. An illustration of soft prompt tuning. Fire icon represents tunable modules and ice icon represents that parameters of those modules are frozen during tuning. Verbalizer are only used for classification task where a mapping from class label to label words is required, which can be one-one mapping, trainable tokens [43], or enhanced with extra knowledge [53].

A general framework of continuous prompt tuning (Figure 6) can be concisely described in the following stages: (1) Given an input sentence $\boldsymbol{c}$ and its corresponding target $\boldsymbol{y}$, a template function $T(\cdot)$ organizes them along with a prompt $\boldsymbol{\tau}$ of length m into a new sentence $T(\boldsymbol{\tau}, \boldsymbol{c}) = \{\boldsymbol{\tau}_{0:i}, \boldsymbol{c}, \boldsymbol{\tau}_{i+1:m}\}$. (2) Subsequently, the sequence $T(\boldsymbol{\tau}, \boldsymbol{c})$ is mapped into an embedding space using model's input layer $\boldsymbol{e}(\cdot)$, resulting in the sequence of token embeddings: $T_e(\boldsymbol{\tau}, \boldsymbol{c}) = \{\boldsymbol{e}(\tau_1), ..., \boldsymbol{e}(\tau_i), \boldsymbol{e}(\omega_1), ..., \boldsymbol{e}(\omega_n), \boldsymbol{e}(\tau_{i+1}), ..., \boldsymbol{e}(\tau_m)\}$, where τ_i is the i-th token in the prompt and $T_e(\cdot)$ denotes the sequence in the embedding space. To perform prompt tuning, $\boldsymbol{\tau}$ is considered as pseudo tokens without explicit semantic meanings, and thus $\boldsymbol{e}(\tau_i)$ is replaced with a trainable tensor $\boldsymbol{h}(\tau_i)$ reparameterized by $\boldsymbol{\theta}^{\tau}$. This modifies the template to: $T_e^{'}(\boldsymbol{\tau}, \boldsymbol{c}) = \{\boldsymbol{h}(\tau_1), ..., \boldsymbol{h}(\tau_i), \boldsymbol{e}(x_1), ..., \boldsymbol{e}(x_n), \boldsymbol{h}(\tau_{i+1}), ..., \boldsymbol{h}(\tau_m)\}$. (3) Finally, we can feed the embedding sequence to an LLM, and optimize the continuous prompts $\boldsymbol{\theta}^{\tau}$ using the downstream loss function $\mathcal{L}$ as follows:

$$\boldsymbol{\theta}^{\tau\star} = \underset{\boldsymbol{\theta}^{\tau}}{arg\ max}\ \mathcal{L}(f_{\Theta}(T_e^{'}(\boldsymbol{\tau}, \boldsymbol{c})), \boldsymbol{y}),$$

where $f_{\Theta}(\cdot)$ is the LLM function parametrized by Θ. For a cloze-style input reformulated from general tasks, for example, the sentiment analysis task for the sentence "I like the movie!" can be rephrased as a cloze-completion problem: "I like the movie! It was [MASK].". The predicted words at the masked position are then employed for subsequent classification. In this case, a unique token [MASK] is integrated during the generation of the template in step (1), and a verbalizer ϕ is required to map class labels to words in the language model's vocabulary, e.g., *positive*⟶ *'great'*, resulting in:

$$\boldsymbol{\theta}^{\tau\star} = \underset{\boldsymbol{\theta}^{\tau}}{arg\ max} \sum_{\boldsymbol{c}} \log P\left([MASK] = \phi(\boldsymbol{y}) | T_e^{'}(\boldsymbol{\tau}, \boldsymbol{c})\right)$$